

What's in my frame?

Point at a photo of the night sky and have something you built tell you what's inside it.

Here's the whole project in a sentence: you take a picture of the night sky, find out exactly where it's pointing, and then build something that can tell you every object sitting inside that picture.

The *where is it pointing* part you'll hand off to a free tool called astrometry.net. It's very good at it, and to be honest it does something you couldn't pull off on your own yet. The *what's inside* part is yours to build — and it's where everything you just learned about databases finally gets pointed at something real.

Why bother building this?

Fair question, and I want you to actually answer it before you write a line of code.

astrometry.net can already hand you a list of objects in your photo. So at first glance the whole project looks pointless — the thing you're about to make already seems to exist.

Go look closer. Hold the list astrometry.net gives you up against what's really in that patch of sky. There's a gap between the two, and that gap is the entire reason this is worth doing. Don't take my word for it — find it yourself.

FIGURE THIS OUT YOURSELF

astrometry.net already names objects in the photo, so what is your database actually for? Don't move past this page until you can answer it in one plain sentence.

The shape of it

You'll build this in small pieces, and you'll keep each piece working before you start the next one. Never let the whole thing be broken at once — that's a miserable way to work and it'll eat your evening. Here's the order I'd go in.

First, decide what a star looks like — to your database

Before you download anything, get a piece of paper. You're about to store a catalog of stars, so think it through: what does one row hold? What columns does it need, and what kind of value goes in each — a number, some text? And if someone tried to add a star with no position at all, should the database allow that or refuse it?

You'll probably get a detail or two wrong here. Leave them wrong. You'll bump into the mistake later, and fixing it then will stick with you far longer than getting it perfect now would.

Then start small, on purpose

Get the Messier catalog — a well-known list of about 110 objects. Load it into your table: read the file, insert the rows. Then write the one query this whole project is built around — given a point in the sky and a distance, hand back everything within that distance of the point.

Run it. It'll be instant. Honestly, it'll feel like you didn't need a database at all, like a plain text file would've done the job just fine. Notice that feeling. It's about to matter.

Now make it real

Swap in a proper catalog. The HYG database holds over a hundred thousand real stars — and later, if you want the point made loudly, you can push to one with millions. Load it exactly the way you loaded Messier.

Now run the same query you already wrote. Don't change a single thing about it.

It crawls.

FIGURE THIS OUT YOURSELF

The query that was instant a minute ago is now slow — and you didn't touch the query. So what changed? Work out what the database is actually doing every single time you ask your question, and roughly how many stars it's checking now versus before. There's a tool in SQL made for exactly this. You've seen it. Find it, use it, run the query again, and watch.

That moment — fast, then painfully slow, then fast again once you fix it — is the most important thing in this whole project. When you reach it, you'll finally get *why* databases are built the way they are. And here's the part I like best: the fix you'll reach for is the same basic idea astrometry.net uses to find your photo in the entire sky. You'll have quietly rebuilt a small version of the trick that makes the big tool work.

Plugging in a real photo

Once the search is quick again, point it at something real. Solve one of your own sky photos on the astrometry.net website. It'll give you back the center of the photo and how wide it is. Feed those two things into your query and read off what's in your frame.

Want a little extra? Instead of copying those numbers across by hand, dig the job's ID out of the results-page web address and have your code fetch the numbers straight from astrometry.net with a single request. That's your first taste of pulling data out of someone else's service in code — one call, nothing fancy.

You're finished when

- you can give it a point and a distance and get back the objects inside that circle;
- it stays quick even on the big catalog;
- you can aim it at a real solved photo and the list it returns makes sense;
- you can explain, out loud and in your own words, why the search got slow and what fixed it.

That last one counts as much as the code. If you can't explain it, you're not done — you've just got it working, which isn't quite the same thing.

If you want to push further

None of this is required. Save it for after the core works.

- Have your program make the whole astrometry.net trip itself — sign in, send the image, wait for the answer — so there's no website clicking at all.
- Aim at a photo near the very top of the sky, or right where the sky's coordinates wrap back around to zero. Something will go wrong with your circle. Working out why teaches you something true about the actual shape of the sky.
- Draw it. Put the objects on a simple map of the frame, the way astrometry.net's annotated images do.

One last thing. The heart of all this is a single query. If you ever catch yourself building something big and tangled, stop — you've wandered off. Keep it small, keep it working, and let the hard part be the thinking, not the typing.